

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	In progress	1
Bug	Operator-blocked	2
Bug	Queued	20
Bug	Ready for testing	1
Task	Operator-blocked	2
Task	Queued	7
TOTAL		33

Items

ATM ID	Type	Status	Severity	Description
LVA-008	Bug	Ready for testing	P1	ROOT CAUSE CORRECTED 2026-08-26 – supersedes the androidx-navigation attrib retained below. C06 + C11 (Challenge11ArchiveOrgAnonymousSearch crashed the app PROCESS at activity-des with IllegalStateException "State must be at least CREATED to be moved to DESTROYED" the inner search/search_input entry. The cause was NOT an upstream AndroidX bug and did not exist in AndroidX code. It was a Lava threading defect: under createAndroidComposeRule, FrameDeferringContinuationInterceptor resumed the Orbit collectSideEffect continuation OFF the main thread, so navHostController.navigate(...) executed on main; navigation-runtime 2.9.1 inserts the back-stack entry BEFORE promoting its lifecycle, and LifecycleRegistry.setCurren

ATM ID	Type	Status	Severity	Description
				throws when invoked off-main — leaving the entry permanently INITIALIZED, which produced the illegal INITIALIZED-to-DESTROYED transition at Activity destroy. FIXED 2026-06-30 by commit ccdd84c1, which wraps the navigate call in runOnUiThread. core/navigation/src/main/kotlin/lava/navigation/NavigationController.kt:101. EVIDENCE CHAIN: last reproduction 2026-06-29 21:00; runOnUiThread guard added 2026-06-29 11:24; the same journey ran clean 2026-06-29 17:48; zero occurrences of the crash signature in any committed evidence since. DISPROVES HYPOTHESIS (retained as forensic record 11.4.6, not erased): the item was attributed to a nested-NavHost lifecycle defect in androidx.navigation (b/244910446 family) and held Operator-blocked pending an upstream repro. Eight app-level candidate fixes were implemented and device-falsified: inner-NavHost Activity-scoped LifecycleOwner, nested-NavHost, ON_STOP force-pop (NavTeardownGuard), nav-compose 2.9.10-to-2.9.8-to-2.10.0-alpha bump, LenientTeardownRule, atomic popUpTo repro, launchSingleTop dedupe, and a full nested-NavHost-architecture collapse. Each reproduction reproduced the identical ISE, and that uniformity was read as proof of an upstream defect. WHY THAT READING WAS WRONG: eight attempts changed WHERE navigate was called or HOW the back stack was structured, but not one changed WHICH THREAD navigation ran on. The off-main call — the actual cause — survived all eight, so all eight reproduced the same crash. Uniform reproduction across all fixes was evidence of an untouched common cause, not of an upstream defect. The filing of a ready upstream repro package at docs/lava/upstream-repro/ was DELETED 2026-08-26; if filing it would have reported to AndroidX, that does not exist in their code. RELATES TO: CLEANUP: the autonomous-QA PASS-overhaul that converted this crash signature into a reported PASS (scripts/autonomous-qa/run-iteration.sh) was removed 2026-08-26 — when the cause fixed, it was a live 6.J bluff mechanism reporting a genuine crash as green. CLOSING: PENDING — THIS ITEM IS NOT CLOSED

ATM ID	Type	Status	Severity	Description
LVA-121	Task	Queued	P3	closure requires a C06 + C11 device re-run HEAD on a 6.AH-conformant container or emulator (never host-direct, never a live A device), both GREEN, with the JUnit XML captured as closure evidence. UNCONFIRMED: this is a HYPOTHESIS, demonstrated defect. It was NOT reproduced during feature 002 and no failing case was constructed. Filed so it is not lost; it must either reproduced (then re-filed as a Bug RED evidence) or explicitly ruled out. CLA scripts/pipeline/phase-01-build.sh counts ARTIFACT lines rather than distinct labels. five duplicate labels would satisfy the >= PASS condition. WHY IT WAS NOT PROMOTED TO A FINDING: not reachable through the sub-scripts, each of which prints a given label most once. To close, either construct a reproducing path or record the argument that none exists. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.
LVA-122	Task	Queued	P3	UNCONFIRMED: this is a HYPOTHESIS, demonstrated defect. It was NOT reproduced during feature 002 and no failing case was constructed. Filed so it is not lost; it must either reproduced (then re-filed as a Bug RED evidence) or explicitly ruled out. CLA the find ... 2>/dev/null inside recompute_evidence_summary in scripts/pipeline/lib/run-report.sh would report zero Evidence Records for an unreadable phase directory, which finalize_run_report would treat as zero rejections. WHY IT WAS NOT PROMOTED TO A FINDING: no realistic reproducing scenario was constructed. To close either construct one or record why the phase directory cannot become unreadable mid-run. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.
LVA-123	Task	Queued	P3	UNCONFIRMED: this is a HYPOTHESIS, demonstrated defect. It was NOT reproduced during feature 002 and no failing case was constructed. Filed so it is not lost; it must either reproduced (then re-filed as a Bug RED evidence) or explicitly ruled out. CLA the compose-file port parse in scripts/pipeline/

ATM ID	Type	Status	Severity	Description
LVA-128	Task	Operator-blocked	P0	phase-04-live-verify-api.sh is a no-op when LAVA_API_LISTEN value in docker-compose.yml is unquoted. WHY IT WAS NOT PROMOTED: A FINDING: the real docker-compose.yml quotes the value, and a wrong port fails curl at curl rather than passing vacuously. To either construct a producing case or make parse fail loudly on an unrecognized shape. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md. WHAT: feature 002 task T054 is a MANDATORY code review of scripts/advance-all-submodules.sh before any invocation against real submodule upstream, per plan.md Hurdle Checkpoint #2. This is the highest-blast-risk new component in the feature: 16-plus submodule upstreams with automatic pin advancement, plus commit-and-push of each submodule own local modifications. CURRENT STATE, stated honestly: the T050-T053 TEST cycle against DISPOSABLE FIXTURE repository COMPLETE - 36/36 assertions green, independently re-run by the orchestrating session - and real submodules were confirmed untouched (git status --porcelain over submodules/ and constitution/ empty, git commit cached empty). scripts/advance-all-submodules.sh has NEVER been run against real submodule or a real upstream. REVIEW MUST COVER at minimum: the GOVERNANCE_DENY default-deny list added by the wave-5 fix and the fact it is deliberately not overridable via LAVA_ADVANCE_SUBMODULES; the 7-step ordering from research.md R-005 (fetch, compare, no-op or checkout-advance, rebased-and-test, discard-and-report on failure, commit push the submodule own local modifications update the parent pin); and the refusal paths REJECTED_BREAKING_CHANGE, REJECTED_PUSH_CONFLICT and REFUSED_GOVERNANCE_DENY. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and specs/002-build-test-distribute-pipeline/tasks.md.
LVA-129	Task	Operator-blocked	P1	WHAT: feature 002 task T062 requires a full end-to-end run of quickstart.md Scenario phases, no shortcuts) on a DISPOSABLE

ATM ID	Type	Status	Severity	Description
LVA-131	Bug	Queued	P0	branch, per plan.md Human Checkpoint # before scripts/pipeline-build-test-distribut trusted to run against real master. CURRENT STATE, stated honestly: the full end-to-end orchestrator run across all five wired phases has NOT been performed. It invokes Gradle systemd and an Android emulator, and was deliberately not run because it is this review gate and because it would have contended with parallel agents already using those toolchains. It is not claimed as done anywhere. SCOPE: NOTE: only phases precondition, build, test, install_boot and live_verify are wired; distro docs and closure are deliberately not wired pending T040/T041 and T048/T049, and a for them via --until is a usage error rather than a silent no-op. Source of truth: specs/002-build-test-distribute-pipeline/progress.yml and 002-build-test-distribute-pipeline/tasks.md WHAT: the phase-02 go test category caused lava-api-go's own §11.4.85 stress and chaos suites to write their evidence into the repository-level .lava-ci-evidence/stress-chaos-jackett/ directory. Six files land there and are TRACKED (confirmed: git ls-files lists exactly 6 paths under that directory). The payload carries a captured_at stamp plus measured latency percentiles, so it differs every run by construction. Net effect: a SUCCESSFUL run left the working tree dirty and the next invocation was refused by the FR-000 precondition, which requires branch master AND a completely clean git status porcelain and exits 2 otherwise. EVIDENCE measured directly: run 2026-08-23T10-17 finalized outcome PASS with phases precondition/build/test all PASS; the immediately following run 2026-08-23T10-31-21Z finalized outcome with a single precondition/FAIL phase, and phase-00 log reads verbatim FR-000: precondition failed – working tree is not This also falsifies quickstart.md Scenario claim that everything a run produces is gitignored. IMPACT: blocks FR-018 and S (a run must not dirty the working tree). STATUS: under active fix in the working tree scripts/pipeline/phase-02-test-go.sh now e LAVA_STRESS_CHAOS_EVIDENCE_DIR

ATM ID	Type	Status	Severity	Description
				pointing at the run's own gitignored evidence directory, and tests/pipeline/test_phase_02_go_evidence_isolation.sh is present as regression coverage; neither is committed yet. CLOSURE CRITERIA: two consecutive full runs both reach their final phase, with the tracked jackett files unchanged after each. WHAT: scripts/pipeline-build-test-distribute installs trap '_close_report' INT TERM (lines 338 and 340) and then invokes each phase as a FOREGROUND pipeline: "\$SCRIPT_DIR/pipeline/{script_name}" "\$run_id" "\$REPO_PATH_OVERRIDE" (lines 342 and 344). Bash does not execute a trap handler when the shell is blocked waiting on a foreground command; the handler is deferred until the command completes. A TERM delivered during a multi-minute Gradle build therefore does not finalize the report at signal time — it finalizes only after the build finishes on its own, which is the opposite of what an interrupt is for. EVIDENCE, measured directly this session, shows a minimal reproduction that mirrors the orchestrator's shape (trap on INT TERM, sleep 6 tee): TERM was delivered at epoch 1787681147 and the handler printed HANDLED at epoch 1787681151 — a 4-second deferral exactly equal to the foreground pipeline's remaining duration. IMPACT: quickstart.md Scenario 5's assertion that a TERM produces a finalized report does not hold while a phase is running, which is precisely when an operator would send one. CLOSURE CRITERIA: the phase runs in the background with the shell waiting on it interruptibly (an equivalent seam), and a test delivers TERM mid-phase and asserts the report is finalized within a bounded interval rather than after the phase completes. WHAT: specs/002-build-test-distribute-pipeline quickstart.md Scenario 3 instructs the regression to check out master, apply a deliberate one-line regression to production code, then invoke scripts/pipeline-build-test-distribute.sh and expect exit code 1 with a test-phase FAIL Evidence Record. That sequence cannot reproduce the test phase: the mutation dirties the working tree, and scripts/pipeline/phase-00-
LVA-132	Bug	Queued	P1	
LVA-133	Bug	Queued	P2	

ATM ID	Type	Status	Severity	Description
LVA-137	Bug	Queued	P2	precondition.sh refuses any run whose git status --porcelain is non-empty, printing FR-000: precondition failed – working tree not clean and exiting 2 (the branch and checkout tree checks are at lines 39-48, both exit 2). The run therefore halts at precondition with evidence never reaches phase-02, and the scenario's stated expected outcome is unreachable. The same quickstart already carries 2026-08-23 corrections to Scenario 3 on two OTHER (absent-versus-SKIPPED phases, and the vacuous zero-Distribution-Records assertion; the document is maintained — this third one is simply not among the corrections. CLOSING CRITERIA: Scenario 3 is rewritten so its falsifiability demonstration is reachable — for example by committing the mutation on a disposable branch, or by using the anti-bluff Evidence Record variant the scenario's opening closing paragraph already describes as available now. WHAT: scripts/hooks/guard-forbidden-commands.sh line 208 gates the §6.T.3 force-push check with <code>[["\$COMMAND" =~ git([:space:]]+^[[:space:]]+)*[:space:] +push]]</code>. The expression is unanchored and <code>([:space:]]+^[[:space:]]+)*</code> group spans arbitrarily many arbitrary tokens, so it matches ANY command in which the literal git occurs anywhere and the token push occurs later, however unrelated the two are. Combined with the standalone <code>-f</code> test on the next lines, a command that also contains a bare minus token (a routine file removal, for instance) is refused as a force-push. EVIDENCE, measured directly this session: the attempt to write <code>ci-evidence/bluff-hunt/2026-08-23-cycle-g-shaping-scripts.json</code> was refused with the verbatim message <code>guardrails: BLOCKED – force-push, because the evidence file's own prose describing this matcher contains the tokens. The write succeeded only by routing around the Bash tool entirely. The same matcher had already blocked the original 2026-08-23 attempt to record that hunt, which is why the evidence file was two days late</code>. ASSESSMENT: the guard fails SAFE — it refuses rather than permits — so no unsafe operation resulted. The cost is different a

ATM ID	Type	Status	Severity	Description
				real: it trains agents to route around a safety guard to get ordinary work done, which enforces the guard's authority for the case it exists to catch. CLOSURE CRITERIA: the matcher is anchored to a genuine command position in a git invocation whose subcommand is push rather than to token co-occurrence anywhere in the string, with fixtures covering both a true force-push and the benign co-occurrence described above. RECURRENCE, 2026-08-26 (this is the pre-existing record; these are additional independent reproductions, filed here rather than as a duplicate item): the same matcher refused an ordinary documentation write TWICE more in a single session, on routine work that involved no version-control operation of any kind. In both instances the refusal was triggered by the DOCUMENT'S OWN PROSE: the text under composition contained the token git, a later occurrence of the token that not the rejected operation, and a bare minus-sign elsewhere in the same string. No command invoked any version-control binary, and neither in either write could have modified any repository. That brings the measured count of benign refusals by this matcher to at least three in two sessions, all of them on evidence-file documentation writes -- which is the population most exposed to it, because documents ALWAYS the guard necessarily quote the very token the guard matches on. WHAT THIS CHANGE ABOUT THE ASSESSMENT: nothing about the direction of failure. The guard still fails SOME and no unsafe operation has resulted. What recurrence establishes is that the false-positive rate on documentation work is not incidental: it is systematic and self-reinforcing, since the attempt to WRITE UP the defect is itself blocked by the defect, which is why the original bug hunt evidence file landed two days late. The cost remains the one already recorded: agents learn to route around a safety guard to complete ordinary work, and a guard routed around has no authority left for the case it exists to catch. The closure criteria above are unchanged; the fixture set should additionally include a benign documentation string that quotes all three trigger tokens and MUST be permitted.

ATM ID	Type	Status	Severity	Description
LVA-148	Bug	Operator-blocked	P2	WHAT: the word channel names two different axes in this feature, and the conflict sits down under the repair path for LVA-120 and for section 6.AA clause 8 condition (D). MEASURED BY DIRECT INSPECTION: specs/002-build-test-distribute-pipeline/contract-distribution-record.schema.json line 11 defines channel as the Firebase APP, with an enumeration the client app distribution target and the app distribution target; scripts/firebase-distribute.sh lines 321 to 324 resolve a channel variable to release or debug, which is the BUILD VARIANT; and scripts/check-cycle-coverage.sh already receives the app-channel through its evidence-directory parameter; channel parameter is a SECOND axis whereas the same word. That parameter is also INCOMPLETE and unvalidated: the variable is set by the argument-parsing branches at lines 37 and asserted non-empty at line 51, and never again in the remaining 128 lines of the script. Both artifacts it checks resolve from the evidence directory and the version alone, debug, release and an arbitrary unknown all produce byte-identical output and exit. RELATED AND SEPARATELY MEASURED: release-channel evidence artifacts exist anywhere. The evidence tree under .lava-evidence/distribute-changelog partitions itself only, into a client directory and an api-app directory, with no debug-versus-release separation any level. Implementing the channel parameter therefore requires first INVENTING what release-channel evidence IS, which is a design decision rather than a code fix. SCOPE Note: this is not read as a duplicate: this item is a vocabulary collision itself together with the absence of release-channel artifacts. The behavioural consequence inside firebase-distribute.sh is tracked at LVA-120, and condition (D) inherits both. DRAFTED OPTIONS: specs/002-build-test-distribute-pipeline/amendment-defects-proposal.md sections 2 and V-5.
LVA-149	Bug	In progress	P0	WHAT: scripts/pipeline-build-test-distribute resolves every git query and every evidence path against the CALLER'S working directory rather than against the repository the run is supposed to be about. Three facts compose

ATM ID	Type	Status	Severity	Description
				the defect, each verified by direct inspection of the script this session. First, the run repository commit identity is taken as <code>commit_sha='\$(git rev-parse HEAD)'</code> with no <code>-C "\$REPO_ROOT"</code> so it names whatever repository the invoking shell happens to be sitting in. Second, <code>RUN_PATH</code> and <code>_run_report_path()</code> are RELATIVE paths under <code>.lava-ci-evidence/pipeline-runs/</code>, so the report is deposited relative to the caller's directory, not the repository under test. Third, the script contains ZERO <code>cd</code> statements, so nothing ever reconciles the two: it never computes <code>\$REPO_ROOT</code>, and it never re-resolves the evidence root against it. <code>REPO_ROOT</code> is computed and then not used for either purpose. MEASURED, not inferred: a run was invoked against a fixture repository, and its FR-00 precondition guard passed legitimately, but the guard inspects the FIXTURE and the repository was on master with a clean tree. The run wrote its report into LAVA'S OWN <code>.lava-ci-evidence/pipeline-runs/</code> tree, carrying <code>commit_sha=545920a160cc8e5591bb4c9c7a87793f377</code> -- Lava's HEAD, a commit the run never built, never tested and never touched -- together with outcome PASS and phases [(precondition, test, PASS)]. IMPACT: the constitution matches evidence to artifacts BY COMMIT SHA in separate places -- 6.Z's same-SHA test-evidence rule, 6.AK's coverage-intersection gate, and 6.AA clause 8 condition (A), which refuses a report whose <code>commit_sha</code> does not equal HEAD at distribute time. A report that names a SHA it never exercised is therefore not mislabelled: it is a false claim at rest, sitting in the exact directory those three gates read, the exact shape they accept. This is the 6.Z -- nothing was learned about that commit, but recorded as though the commit passed. CONTAINMENT ALREADY PERFORMED: such artifacts were preserved to scratchpads for forensics and removed from the evidence base. the 57 pre-existing legitimate reports were untouched and were not modified in any way. STATUS: a fix is IN FLIGHT with another report in this session, which is why this item is flagged in progress rather than Queued -- it is recorded here so the defect cannot evaporate if that

ATM ID	Type	Status	Severity	Description
LVA-151	Task	Queued	P1	is interrupted (section 11.4.197). CLOSURE CRITERIA: every git invocation in the orchestrator is repository-scoped via -C "\$REPO_ROOT" (or the script enters \$REPO_ROOT before any of them), the evidence root is resolved as an absolute path beneath \$REPO_ROOT, and a regression test involving the orchestrator from a DIFFERENT working directory than the repository under test asserts both that the report lands in the target repository's evidence tree and that its commit_sha equals the target repository's HEAD rather than the caller's. WHAT: multiple agents working concurrently under ONE shared working tree cause constituent gates to report failures that are artifacts of contention rather than properties of the target. FIVE separate gate failures during this session, each dissolved on isolated re-measurement. No code change was required for any of the MEASURED CASES, each re-measured in isolation before being dismissed: (a) a workflow items check reported as a timeout was slow under concurrent load, not a hang -- it completed normally once the tree was quiet; (b) a covenant-anchor check reported as a race caused by the MEASURING process's own timeout terminating a probe part-way through mutation, so the measurement destroyed the state it was measuring; (c) a process-liveness check reported a stale runner that was the pgrep pattern matching the checking process itself, a self-match; (d) verify-all's markdown export-sync gate failed because a different agent was regenerating .html and .pdf siblings during the sweep, transiently leaving a .m newer than its own exports -- the isolated runner reported 282 files examined, 0 problems, and was repeated four consecutive times with the identical result, which makes the true verify-all figure 58 of 58 rather than the 5 with-1-failure the contended sweep produced; (e) check_constitution_test.sh's test_live_repo_passes failed twice under 1 and PASSED in isolation with exit 0 and 5 cases. COST: five separate diagnostic efforts were spent establishing that nothing was wrong, and in case (d) a contended sweep produced a headline number that understated

ATM ID	Type	Status	Severity	Description
LVA-152	Bug	Queued	P2	the tree's real compliance -- a false RED, erodes trust in the gates in exactly the way a false GREEN erodes trust in the tests. IMPLICATION EVIDENCE: an attestation captured during contention records a verdict that is a product of the scheduling, not of the tree, so it is not reproducible and cannot serve as evidence either way. THE RULE THIS ITEM EXISTS IN THE RECORD, stated plainly so a sixth diagnosis was not spent rediscovering it: A CONSTITUTIONAL-GATE FAILURE IS NOT A FINDING UNTIL IT REPRODUCES IN ISOLATION. A gate result captured while agents are writing to the tree is provisionally promoted to a finding only by a quiet-tree re-run, and it is that re-run's output that belongs in an evidence file. CLOSURE CRITERIA: the problem above is documented where gate-runners read it, and either a lock file serializes gate invocations across concurrent agents or a long-running gate acquires an advisory lock for the duration of its sweep, with a test proving that two concurrent invocations serialize rather than interleave. WHAT: tests/pipeline/test_advance_all_submodules.sh defines a helper, run_script, whose body assigns the script's captured output to the GLOBAL variable LAST_OUTPUT so that failure branches can print what the script actually said. Every existing caller in cases 1 through 5 invokes helper inside a command substitution, in shape "\$ (run_script ...)". A command substitution executes in a SUBSHELL, and a subshell's variable assignments do not propagate to the parent shell. LAST_OUTPUT in the calling shell therefore never receives the value the helper assigned: it holds whatever was held before -- an empty string on the first run, and thereafter the value left by whichever earlier assignment last ran in the parent shell. CONSEQUENCE: every FAIL-branch diagnosis in cases 1 through 5 that prints LAST_OUTPUT would print a stale value rather than the current value of the invocation that just failed. WHY IT HAS NEVER BEEN VISIBLE: those FAIL branches do not currently fire -- the suite passes -- so the stale value is never printed and nothing surfaces the defect. It is latent by construction.

ATM ID	Type	Status	Severity	Description
				and will surface at exactly the worst moment when a genuine regression trips one of the branches and the operator reads a diagnostic describing a DIFFERENT invocation than the one that failed. SEVERITY RATIONALE, PARTIAL STATE: the assertion is wrong and no result is falsified because the suite's verdicts are correct. What is wrong is the diagnostic attached to a future failure rather than the cost is paid in misdirected debugging rather than in a wrong pass. PARTIAL STATE: ALREADY IN THE TREE: case 6, added during this session, does not use the helper's global pattern -- it captures the script's output in the CURRENT shell instead, which is the correct pattern and can serve as the model. The cases in cases 1 through 5 are unfixed. CLOSURE CRITERIA: run_script either returns its output on stdout and every caller captures it in the current shell (the case 6 pattern), or the helper is invoked without a command substitution and its global assignment reaches the caller; a deliberately-failing fixture asserts that the printed diagnostic contains a string unique to the failing invocation, which the current closure cannot satisfy. WHAT: the LVA-148 axis rename established 'app' (Firebase app: client vs api-app) and 'build_variant' (release vs debug) as the two distinct names. CHANGELOG_CHANNEL was deliberately NOT renamed, because script pipeline/phase-05a-changelog-entry.sh:30 greps scripts/firebase-distribute.sh for the LITERAL string CHANGELOG_CHANNEL and two tests/pipeline/ files mirror that literal. Renaming it in firebase-distribute.sh alone would break a live gate at a distance, which is why the rename agent stopped rather than proceeding. OWED: one ATOMIC rename CHANGELOG_CHANNEL -> CHANGELOG_CHANNEL across 4 files / 5 sites, landed together. CONTAINED MEANWHILE: test case P5 shows that the variable can only ever hold an app slug and cannot silently acquire a build-variant value while the rename is pending. Source: self-discovered during the LVA-148 rename, 2026-08-26.
LVA-153	Task	Queued	P2	
LVA-154	Task	Queued	P2	WHAT: the LVA-148 rename separated 'app' from 'build_variant' in the contracts and source but the prose that describes them was not

ATM ID	Type	Status	Severity	Description
LVA-155	Bug	Queued	P1	updated in the same pass. Affected: CLAUDE.md 6.AA clause 8 still calls the b variant axis a channel; specs/002-build-test-distribute-pipeline/spec.md likewise; and 002-build-test-distribute-pipeline/data-model.md's table now DESCRIBES A PROPERTY THAT NO LONGER EXISTS, s distribution-record.schema.json's channel renamed outright with no alias (0 records rest, so nothing became unreadable). Also at the same time: scripts/tag.sh:285-306 u channel_label / channel_dir for a THIRD s of the word, and scripts/pipeline/phase-02 hermetic.sh:72-75 cites tests/firebase/repro_mode_both_channel_gap.sh, which deleted earlier the same day when combin distribute mode was retired. WHY IT MAT documentation describing a property that not exist sends the next reader to look for and the whole point of the rename was th word naming two axes is a trap. Governar edits require operator review, which is wh is filed rather than done. Source: self-discovered during the LVA-148 rename, 2026-08-26. WHAT: .githooks/pre-push Check 10 invol scripts/check-cycle-coverage.sh with --head=\$sha where \$sha IS the commit advancing the distribute pointer. The evid that commit references was written BEFO existed, so it cannot declare that commit' SHA. Now that the LVA-149/F2 fix makes commit-SHA binding actually bind (it was which is why 5 of 9 shipped cycles passed against an arbitrary wrong HEAD), this se reference surfaces: the next pointer-adva commit will be REJECTED at push time. MEASURED: the binding fix took wrong-H acceptance from 5/9 to 0/9; before the fix rejection was hidden. NOT closed by the t agent, deliberately and correctly: it declin implement an ancestor/code-identity esca because it could not prove the acceptance with a hermetic positive case, and an unt acceptance path inside a safety gate is pr the defect class it was dispatched to remo Three candidate options are written up in evidence log. DOES NOT BLOCK the curr push: Check 10 fires only on commits tha

ATM ID	Type	Status	Severity	Description
				advance a distribute pointer, and the penultimate change set touches no last-version or distribute changelog path (verified). Source: self-discovered during the LVA-149 fix, 2026-08-26. Needs an operator decision before the next distribute. WHAT: re-running the REPAIRED cycle-coverage gate over all 9 shipped cycles should hold up on substance — the gate was broken but those releases were not shipped on no 1.3.12-1077 is the exception and it FAILS the fixed gate: 2 of 3 claims uncovered, any of the 13 covering refs its cycle-coverage asserts have NO verdict record in the evidence at all. One asserted ref is the prose 'Full Challenge suite proven on Genymotion' standing in for 9 named Challenges. This is exactly the shape of the 2026-06-26 incident (.lava-ci-evidence/sixth-law-incidents/2026-06-26-c00-only-gate-shipped-broken-flows.json) that 6.AK was written to prevent recurring ONE CYCLE after the clause last — because the gate that was supposed to do it could not read the evidence format the one used. Two further cycles, 1.3.17-1084 and 1.3.17-1085, declare NO commit SHA at all their evidence. NOT a live regression: it is a historical finding about what was actually verified versus what was reported. Record the record of that release is honest. Source: self-discovered during the LVA-149 blast-radius assessment, 2026-08-26. WHAT: the three suites asserting 6.AA clause (test_phase_05_distribute_gate.sh, _vacuum_fix_audit.sh) each call write_evidence_record once and validate_evidence_record ZERO. Until 2026-08-26, write_evidence_record stamped every record with anti_bluff_status 'validated' — the SAME value the independent validator writes on accept — so a record not examined was byte-identical on disk to one examined and accepted, and every consumer read that field as a verdict. The aggregate compounded it by treating an absent status as 'not rejected', an asymmetry against 'results' where an unrecognised value already fell through to FAIL. NET EFFECT: clause 8 condition (B) — 'zero Evidence Records contain anti_bluff_status other than validated' — v
LVA-156	Bug	Queued	P1	
LVA-157	Bug	Queued	P0	

ATM ID	Type	Status	Severity	Description
LVA-158	Bug	Queued	P1	satisfiable by records that were never valid. INCLUDING all four mandatory real-device challenge records for app-debug, app-release, api-app-debug and api-app-release, every one of them Challenge00CrashSurvival. Clause 8 rule authorising a distribution to proceed without debug to release WITHOUT an operator present. WHY THIS IS P0: that is the 2026-06-26 'only gate shipped broken flows' forensic analysis (.lava-ci-evidence/sixth-law-incidents/) reproduced ONE LEVEL ABOVE where the sweep found it — not in a release's evidence but in the test fixture that CERTIFIES the authorising unattended release. 6.Z clause states a cold-start check is the minimum and explicitly NOT sufficient. ATTRIBUTION SETTLED by controlled A/B, not inferred: swapping only evidence.sh + phase-02-test gives pristine -> all three EXIT=0, fixed - three EXIT=1; the suites were relying on fail-open default. UNDERLYING FIX LAVA: the writer now emits a schema-legal fail-c placeholder and the aggregator classifies ways (validated / REJECTED* / anything else UNEVALUATED, which fails the phase). Once the fixtures need repair, and it is not yet established whether clause 8 can still qualify C00-only evidence once they validate properly — that is being determined now. Source: discovered during the tag.sh critical fail-c fixes, 2026-08-26. WHAT: .gitmodules declares 'ignore = untracked' for submodules, so 'git status' parent does not report untracked files left inside a submodule working tree. The pipeline FR-000 precondition guard reads that status decide the tree is clean, and therefore P0 with untracked leftovers present in two submodules. STATUS CHANGE: the 2026 vacuum-pass sweep recorded this as P1 UNCONFIRMED; it is now CONFIRMED. FIXTURE during the corpus-floor sweep. IT MATTERS: FR-000 exists so the pipeline never builds, tests or distributes from a tree carrying uncommitted state — a run that verifies artifacts built from unrecorded local changes attributes its evidence to a commit does not describe what was actually built is the same attribution failure class as LVA-

ATM ID	Type	Status	Severity	Description
				(a run report claiming a commit_sha the n never tested), reached by a different route. Composes with the T054 re-review finding every push and both B-2 residuals in advance all-submodules.sh sit behind has_local_m a precondition the script neither checks n states. Source: self-discovered, sweep fine P11 promoted from UNCONFIRMED to CONFIRMED, 2026-08-26. WHAT: the test's 'git' stub does not honour -z flag, so the file list it feeds the three se 6.R scanners (scan-no-hardcoded-uuid.sh, no-hardcoded-ipv4.sh, scan-no-hardcoded hostport.sh) is not parsed as the scanners expect, and they examine ZERO files. The has therefore been green while asserting nothing about credential or hardcoded-lit detection. PROVED BY HEXDUMP during 2026-08-26 corpus-floor sweep; it became visible only because the new corpus floor a zero-file scan refuse instead of silently succeeding. SCOPE, stated precisely: the measured is in the TEST'S STUB, so what established is that the TEST was vacuous NOT that the production scanners are bro against a real corpus. Confirming product behaviour is separate work and is NOT cl here. WHY IT MATTERS: section 6.H treat credential leakage as a security incident a section 6.R forbids hardcoded literals; a s that certifies those scanners while feeding nothing is the highest-consequence instan the vacuous-pass family found this cycle. FIXED: the file is staged by another agent concurrent work; the remedy is one line i stub. Source: self-discovered during the c floor sweep, 2026-08-26. WHAT, MEASURED not argued (CASE 19 in tests/pipeline/ test_phase_05_distribute_gate.sh, re-meas every run rather than asserted in prose): 'records naming a Challenge OTHER than Challenge00CrashSurvival: 0', the gate re 'CONDITION (C) Device evidence for BOT variants : PASS' and 'clause 8 QUALIFIES 3)'. Condition (C) at scripts/pipeline/phase distribute.sh:908-1022 checks variant bin category, result, anti_bluff_status and the container-or-VM runner — it NEVER read
LVA-159	Bug	Queued	P1	
LVA-160	Bug	Queued	P0	

ATM ID	Type	Status	Severity	Description
				test_id. WHY THIS IS P0: clause 8 is the n authorising a distribution to proceed from debug to release WITHOUT an operator p and clause 8(C) states IN ITS OWN TEXT cold-start 'does not by itself satisfy 6.AK'. implementation does not enforce what the asserts. This is the 2026-06-26 'C00-only shipped broken flows' incident condition, reachable through the gate written to pre it. AGGRAVATED BY THE 2026-08-26 AMENDMENT: the re-lettering withdrew condition (D) — the only condition gestur per-variant channel verification — record as 6.AA-pipeline-debt-D OWED rather than replacing it, on an analysis stating that '(C becomes MORE load-bearing, not less: wi retired it is the sole mechanical release-v guarantee'. That was true of the STRUCT and false of the SUBSTANCE, and the assumption was not tested before the amendment was applied. THE FIX (specif NOT implemented — the gate was deliber not touched): require, per variant, at leas PASSing real-device-challenge record who test_id is NOT the cold-start canary — equivalently, intersect the executed-PASS Challenge set with the cycle's claim-set p 6.AK clause 1. Nothing implements this to Source: self-discovered during the clause fixture repair, 2026-08-26. WHAT, measured live during the T062 end run (run_id 2026-08-26T14-09-17Z): t test phase reported 74 real-device-challen FAILs. They are a TIMEOUT ARTEFACT, n broken features. phase-02-test-challenge. selects 73 Challenge classes in a SINGLE emulator-matrix invocation, and NEITHER NOR scripts/run-challenge-matrix.sh pass test-timeout (verified: zero occurrences in either file), so cmd/emulator-matrix's DEF 10 minutes applied. Attestation row: test_seconds=600.02, test_error='signal: Gradle's last log line: 'Tests 81/104 compl (6 skipped) (12 failed)'. Gradle was killed writing JUnit XML, so the phase parsed 0 and marked all 73 classes FAIL. Of what actually ran, 12 genuinely failed (e.g. Challenge46SearchTimeoutInterruptTest ComposeTimeoutException after 15000 m
LVA-161	Bug	Queued	P1	

ATM ID	Type	Status	Severity	Description
LVA-162	Bug	Queued	P1	UNCONFIRMED: the true status of the remaining ~61 — settled by raising --test-timeout or batching the classes. The api-a half is a SEPARATE, GENUINE failure (test_seconds=124.88, exit status 1), not a timeout. WHY IT MATTERS beyond the w run: a future operator reading this evidence would conclude the product is broken when not, and the same reasoning error in reverse: how a real breakage gets dismissed as 'just a timeout'. WHAT THE PIPELINE GOT RIGHT: worth preserving: it fabricated no passes for killed classes, wording them 'No entry found for 0 file(s) parsed — either the class filter did not match, or the run aborted before instrumentation completed'. Source: self-discovered during the T062 run, 2026-08-26. WHAT, measured during the T062 end-to-end run: the per-AVD attestation row is marked as gating: true but carries diag = {} and failure_summaries = []. Section 6.I.4 (Group-Extension) requires every row to carry diag.target / diag.sdk / diag.device / diag.adb_devices_state — the per-AVD for the snapshot captured immediately before instrumentation — plus parsed failure_summaries. scripts/tag.sh Group-Extension 3 exists to catch the AVD-SHADOW BLUEPRINTS comparing diag.sdk against api_level and refusing when they differ; with diag empty, the comparison has nothing to read. The gate is therefore inert on exactly the rows it is meant to police, and it is inert on a row that declares itself gating. This is the vacuous-pass family of gate reaching a verdict having examined no items) landing inside the emulator attestation path, which the 2026-08-26 sweep did not catch because it swept scripts rather than emitted evidence. Composes with LVA-160 (clause 6.I.4 condition (C) never reads WHICH Challenge ran): two independent device-evidence gates, each unable to see the field it exists to check. Source: self-discovered during the T062 run, 2026-08-26. WHAT, measured 2026-08-26 after initialising the previously-uninitialised nested submodule at their already-recorded pins (zero pins moved): the recursive submodule set carries 41 remotes, of which 41 are own-org (vasic-d
LVA-163	Task	Queued	P1	

ATM ID	Type	Status	Severity	Description
				HelixDevelopment) and 28 are THIRD-PARTY. They include github.com/google/perfetto, bytedance/UI-TARS and UI-TARS-desktop-square/leakcanary, appium/appium, Genymobile/scrcpy, chroma-core/chroma, framework/allure2, anthropics/anthropic-quickstarts, SigNoz/signoz, Skyvern-AI/skyvern, run-llama/llama_index, mem0ai/mem0, kiwitcms/Kiwi and 15 more - nearly all verifiable under submodules/helixqa/tools/opensource. PLUS THE TOP-LEVEL SUBMODULE NAME 'superspec', which points at github.com/WangX0111/superspec. IMPACT: the operator directive 'commit and push all Submodule recursively to all upstreams', executed literally, would attempt a push to 28 repositories owned by other people and organisations. Such attempts would almost certainly fail for lack of write access, but attempting them is wrong. If a credential that DID grant access would be used, it an incident rather than a failed command. CONVERGING MEASUREMENT from the design pass: the 27 previously-uninitialised submodules are EXACTLY submodules/helixqa/tools/**, and every one has a third-party upstream - so they are precisely the 27 third-party CLAUDE.md's Automated Pipeline Pin-Addition Path condition (C) forbids advancing. Initiating them makes them RECORDABLE, never ADVANCEABLE. NOTE the governance decision deny introduced by LVA-138 is what contains this today: superspec was one of the several submodules named by NO operator decision, and it points at a third-party repository - the allow-list defaulting to deny is doing real work. OPERATOR DECISION 2026-08-26: push to github.com remotes ONLY, and REFUSE each third-party remote with a named per-remote record rather than skipping it silently - 'all submodules handled' without per-submodule records is not evidence, by the same reasoning section 6.W applies to per-AVD attestation rows. Alignment section 6.W (which governs where WE push) and with condition (C). FILING NOTE: the previous version of this record was mangled by unescaped backticks in the authoring shell command, which silently dropped the submodule name in two places. Corrected and recorded because a tracker entry that quipped

ATM ID	Type	Status	Severity	Description
LVA-164	Bug	Queued	P1	lost a fact is the same defect class this cycle has been evicting all day. WHAT, measured 2026-08-26 immediately after initialising the 29 nested submodules at the top of already-recorded pins: submodules/helixqa. Helixqa reports a permanent dirty state at the parent level (M submodules/helixqa), caused by a trailing ending artifact three levels down. CHAIN, traced end to end: helixqa -> tools/opensource/docling (gitlink pin 4f37eeee == actual H 4f37eeee, so NO pin moved) -> tests/data/sources/pftaps057006474.txt. That file shows with MIXED line endings (825 CRLF among 1380 lines, 75716 bytes) while docling's own .gitattributes declares '* text=auto', which normalises on checkout and the worktree and index never match the index. Git states the mechanism itself: 'CRLF will be replaced by the next time Git touches it'. DECISIVE TEST: stripping CR from both the index blob and the worktree file yields byte-identical content; the difference is line endings ONLY - there is no content change and no pin move. NOTE on instruments: 'git diff --ignore-cr-at-eol' does NOT neutralise this and reports 1 difference because the endings are mixed rather than uniformly CRLF-at-EOL. An earlier check that flag as the discriminator reached the conclusion; the correct discriminator is a stripped byte comparison. Recorded because using the wrong instrument and believing the answer is the defect class this cycle has been evicting. WHY THIS BLOCKS: submodule helixqa is the ONLY submodule the governance allow-list permits advancing (LVA-138 involved the list to default-deny with helixqa as the entry), and the T054 round-2 fix made an unclean submodule working tree a REFUSAL default, opt-in only via a flag the pipeline passes. So the one advanceable submodule is now un-advanceable. It also fails the pipeline FR-000 clean-tree precondition at the parent level. Neither was true before the init. REPRODUCIBILITY: inherent to docling@4f37eeee and its .gitattributes; it reproduces on any fresh clone on a platform where git normalises. It is not local damage; must not be 'fixed' by committing the normalised file into a third-party submodule.

ATM ID	Type	Status	Severity	Description
LVA-165	Bug	Queued	P0	do not own. CANDIDATE REMEDIES, none chosen: (a) set core.autocrlf=false / core. for that submodule and re-check-out so the worktree matches the index; (b) add a local info/attributes marking that path binary stops normalising; (c) teach the clean-tree precondition to treat an eol-only difference clean - REJECTED as a default because it weakens the very check that makes the safe, and would need to be provably scoped eol-only; (d) do not initialise docling at all which conflicts with the operator's all-59-recursive decision. Source: self-discovered while verifying the consequences of the recursive init, 2026-08-26. WHAT: during the T054 architectural analysis an agent wrote a shell expansion of the COLON-DOLLAR-BRACE GITLAB_TOKEN COLON-DASH NO BRACE, intending a presence check. That operator expands to the variable's value when the variable is set, so a real GitLab personal-access token was printed into the session transcript. CONTAINMENT, verification measurement not assertion: 0 tracked files match a token pattern (git ls-files piped to grep), and 0 scratchpad artefacts contain unredacted token. The token did NOT read THIS IS A REPEAT. CLAUDE.md's 67th section-6.L cycle records the IDENTICAL mistake with LAVA_FIREBASE_TOKEN, fixed in LVA-5, where the operator chose to accept risk rather than rotate. The same footgun now fired twice in one codebase, in two different agents, months apart. That is the signal worth acting on: it is not an individual lapse, it is a shell idiom that reads like a presence-check and is not one. OPERATOR ACTION REQUIRED per section 6.H clause: rotate the GitLab token. REMEDY WORTH CONSIDERING beyond rotation: a presence check must never use the COLON-DASH operator. The safe forms are a test on the variable being non-empty, or printing only a fixed literal such as SET or UNSET. A grep over tracked scripts and agent-authored commands for COLON-DASH applied to a name matching TOKEN, SECRET, KEY, PASSWORD or CREDENTIAL would catch the class mechanically rather than relying on

ATM ID	Type	Status	Severity	Description
LVA-166	Bug	Queued	P0	nobody making the mistake a third time. Source: self-discovered and self-reported acting agent, 2026-08-26. WHAT, proven against the current file: the operating precondition stated in the script's own header at lines 116-125 says that with a CLEAN submodule tree the run is 'all local and undoable, nothing leaving this machine'. 'FALSE. PUBLISH_LOCAL_MODS is last consulted at line 1810; the step-5 verify command is invoked later and is NOT gated by it, so the verify step executes on the DEFAULT, FLAGLESS, CLEAN-TREE path with the operator's live credentials in reach. The agent demonstrated the capability by performing an SSH-authenticated git push as an ordinary child process (target was a local bare repo over ssh-localhost; no real upstream was contacted). The operator's SSH key is unencrypted on disk. 25 own-org upstreams are reachable. WHY THIS MATTERS MORE THAN ANY SINGLE ESCAPE: review rounds 4 and 5 both APPROVED the script conditional on passing --publish-local-modifications, on the stated basis that the clean-tree path creates a commit and issues no push and had 'survived two full adversarial passes'. That conclusion rests on a safety property the code does not implement. Five review rounds and eleven proven escapes were all reasoned about within a boundary that does not exist. ADDITIONAL MEASURED FACTS from the same analysis: (1) environment scrubbing does NOT close the 'env -i' with HOME redirected to an empty directory STILL pushed successfully, because OpenSSH resolves the tilde from the password entry rather than from HOME; (2) path manipulation DOES work - bwrap with HOME and /run tmpfs made both push routes fail while leaving network intact; (3) isolation closes ZERO of eleven escapes, proven rather than argued because a fully sandboxed verify step still poisoned the commit headers - the SCRIPT publishes afterwards; what isolation removed review 5's strongest argument for the future guarding. RESIDUAL, stated honestly: a sandbox still has write access to the tree it builds, so it can lie about the build; cache poisoning across the boundary is real and

ATM ID	Type	Status	Severity	Description
				unaddressed since the gradle cache must be writable and a later unsandboxed build exports what it finds. UNCONFIRMED: whether a Lava build completes inside the sandbox - experiment gates the whole item. Source: discovered during the T054 architectural analysis, 2026-08-26. WHAT: the credential-masking sandbox prototyped for LVA-166 is a GIT-CREDENTIALS sandbox, not a credential sandbox, and the difference is load-bearing. Masking HOMERUN/user removes the SSH agent socket and on-disk key, which provably blocks git push (probe fired 45s into a live 17-minute build; both routes exit 128, zero refs; falsified by unsandboxed control that succeeds, and by removing only the two tmpfs lines, after which both pushes land). But the build must be outside the repository tree, and that tree carries secrets: .env (4072 bytes - Firebase token, HMAC secret, tracker credentials), app/google-services.json and api-app/google-services.json and keystores/release.keystore plus keystore.debug.keystore. All are gitignored, so secrets.6.H holds with respect to git - but gitignores does not withhold them from the build. PROVEN, not argued: the SANDBOXED build ran uploadCrashlyticsMappingFileRelease TWICE, i.e. it reached Firebase from inside the sandbox. So the claim 'nothing leaving the machine' remains FALSE even with credential masking in place. Whoever corrects the false header recorded in LVA-166 must not substitute a smaller falsehood. CONTEXT - the sandbox itself WORKS and this is not an argument against it: a real 4-variant Lava Android build completed inside it (BUILD SUCCESSFUL 17m 50s, exit 0, all four APKs apksigner-verified with real Lava certs, 3-ABI Go+cgo liblava cross-compiled through the NDK), a cold-rebuild was byte-identical to the host's, and the cache reuse survives (2682/3035 tasks UNDATE, ~54ms fixed entry cost, A/B wall-time deltas of +42/+46/-42s on a ~130s build - flips, indistinguishable from noise). TWO FURTHER RESULTS worth keeping: (a) the architecture analysis's own prototype mask run WHOLESALE, but this repository lives in run/media/... - following it literally deletes
LVA-167	Bug	Queued	P1	

ATM ID	Type	Status	Severity	Description
LVA-168	Bug	Queued	P0	project and fails in 0.01s with 'cd: No such file or directory'. An implementer following it have recorded a FALSE NEGATIVE and discarded a working control. Mask /run/never /run. (b) the cache-poisoning residue analysis disclosed and could not close IS closable: --ro-bind on the gradle cache fail Gradle WRAPPER needs a writable .lck and build dies at 0s), but bwrap --overlay-src v tmp-overlay works - build exit 0, identical host cache verifiably unpoisoned, push still blocked, at roughly 3x warm wall time. The poisoning was confirmed end-to-end first: sandboxed process writes gradle init.d/ and later UNSANDBOXED Gradle executes it with the agent socket present. NOT ESTABLISHED only the BUILD half was measured. phase test.sh under masking is UNCONFIRMED the higher-risk half, because section 6.AH emulators interact with the mount names. Source: self-discovered during the LVA-16 sandbox feasibility experiment, 2026-08-2 WHAT, measured live during T062 attempt the pipeline run report never discloses a SKIPPED phase. A run invoked with --skip omits 'test' from phases[] ENTIRELY, so the report shows outcome PASS with every list entry PASS - a run that executed ZERO test byte-indistinguishable in its report from one that ran the full suite and passed. Captured 2026-08-27T00-15-42Z, exit 0, precondition PASS / build PASS / test SKIPPED / install PASS / live_verify PASS, with no 'test' entry in phases[] at all. WHY THIS IS SERIOUS: section 6.AA clause 8 condition (B) requires that every entry in phases[] is PASS and that zero Evidence Records carry a non-validated anti_bluff status. A run that SKIPPED the phase which produces nearly all Evidence Records satisfies (B) vacuously - the array is all PASS because the phase that could have failed is not in it. This is the vacuous-pass family, at the run-report level inside the artifact that clause 8 reads to authorise an unattended debug-to-release distribution. It composes with LVA-160 (condition (C) never reads WHICH Challenge ran) and LVA-157 (clause 8's own certifying fixture never validated their records): three independent ways the same clause can qu

ATM ID	Type	Status	Severity	Description
				on evidence that proves nothing. THE FIX skipped phase MUST appear in phases[] v explicit SKIPPED result, and clause 8's evaluation must treat a SKIPPED phase as disqualifying rather than absent. An omitted phase and a passed phase must never be the same bytes. SELF-CORRECTION recorded the measuring agent: it had earlier asserted that a --skip run 'can never report outcome PASS', taken from a code comment rather than from measurement. The measurement disproved the comment. Recorded because trusting a comment over an experiment is a defect class this cycle has been evicting. Source: self-discovered during the T062 end run, 2026-08-27. WHAT, measured during T062 attempt #4 scripts/pipeline/phase-02-test.sh writes lava-go/internal/*/lava-ci-evidence/ directories are neither tracked nor gitignored. The pipeline's own FR-000 precondition requires a clean working tree, so once the test phase run, EVERY subsequent run in that check refused - the pipeline dirties the tree it is to test and then cannot start again. This is identical shape as the pipeline-runs/ ignored defect closed earlier this cycle by a prevent-check-ignore guard, reached by a different that guard probes .lava-ci-evidence/pipeline-runs/ specifically and does not see these per-package directories. Source: self-discovered during the T062 end-to-end run, 2026-08- WHAT, measured during T062 attempt #4 LVA-162 fix added a Lava-side refusal for gating attestation row carrying an empty and it correctly FIRED in phase-02 on live phase-04 accepts the same empty-diag row the same frozen submodule emitter (submodules/containers pkg/emulator make writeAttestation). So one phase enforces 6.I.4's diag requirement and another does on identical input. A gate that is enforced on one path and not another is worse than one enforced nowhere, because its presence is coverage it does not provide. Source: self-discovered during the T062 end-to-end run 2026-08-27.
LVA-169	Bug	Queued	P1	
LVA-170	Bug	Queued	P2	
LVA-5	Bug		P0	

ATM ID	Type	Status	Severity	Description
		Operator-blocked		67th-cycle: the LAVA_FIREBASE_TOKEN default-expansion printed the token to the session transcript (NOT committed to git). Operator MUST rotate per §6.H clause 6 (firebase logout; firebase login:ci). Source discovered — .lava-ci-evidence/sixth-law-incidents/2026-05-20-firebase-token-echo-leak.json OPERATOR DECISION, 2026-08 ACCEPTED RISK. The operator has reviewed this item and elected to ACCEPT the risk than rotate the credential at this time. This is therefore NOT abandoned work and NOT pending action awaiting an agent: it is a recorded risk-acceptance, retained in the register so it stays visible rather than being closed and forgotten. WHAT IS AND IS NOT TRUE AS A RESULT: the leak itself remains a matter of record — the token value was expanded into a session transcript on 2026-05-20 and was never committed to git. The incident record cited above — and the decision does not retroactively make that exposure harmless. What the decision sets the RESPONSE: no rotation is scheduled, no agent should treat the absence of rotation as an oversight or re-raise it as a new finding. WHY THIS ITEM STAYS OPEN RATHER THAN CLOSING: closing it would assert the underlying condition was remediated, which is not what happened, and section 6.J forbids recording an outcome the work did not produce. An accepted risk that has been closed is indistinguishable in the tracker from a risk that was fixed, and that is precisely the conflation this register exists to prevent. RE-OPENING CONDITION: if the accepted token is observed authenticating after the operator's intent changes, or if the operator later elects to rotate, this item resumes as ordinary pending work under its original unblock options [A], [B] unchanged.